Multimedia Tools and Applications (2024) 83:14011–14045
https://doi.org/10.1007/s11042-023-15408-8

OMRNet: A lightweight deep learning model for optical mark recognition

Sayan Mondal[1] · **Pratyay De**[1] · **Samir Malakar**[2] · **Ram Sarkar**[3]

Received: 26 February 2022 / Revised: 9 February 2023 / Accepted: 18 April 2023 /
Published online: 12 July 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
Existing Optical Mark Recognition (OMR) systems tend to be expensive and rigid in their operation, often resulting in erroneous evaluations due to strict correction protocols. This scenario airs the need for a flexible OMR system. Hence, in this work, we propose a lightweight transfer learning based Convolutional Neural Network (CNN) model, dubbed as OMRNet, which can classify answer boxes on any generalized OMR test sheet. Unlike most existing techniques that rely on image processing algorithms to recognize extracted answer boxes in two classes: confirmed and empty, the OMRNet is designed to classify the answer boxes into confirmed, crossed-out, and empty categories. That is, OMRNet is facilitating the crossing out of previously answered questions and thus removing the rigidity of templates in Multiple Choice Question (MCQ) tests. We have built OMRNet on top of a MobileNetV2 backbone connected to four fully connected layers with appropriate dropouts and activation functions in between. We have evaluated OMRNet on the Multiple Choice Answer Boxes dataset available at https://sites.google.com/view/mcq-dataset. We have performed experiments following a 5 fold cross validation scheme, and OMRNet has achieved accuracies of 95.29%, 95.88%, 93.97%, 97.45%, and 97.20%, with an average accuracy of 95.96%. Also, the experimental results confirm that the present model performs better than the compared state-of-the-art methods and standard CNN models in terms of accuracy, execution time, and memory required to store the trained module. Moreover, we have employed a quantization technique to make the trained module more memory efficient and deployed it to a web app using our own Representational State Transfer Application Programming Interface (REST API). It makes OMRNet available via a Hypertext Transfer Protocol (HTTP) endpoint, allowing potential users to connect to it via the Internet. The source code for the work is available at the following link: https://github.com/sa-y-an/OMRNet.

Keywords Optical mark recognition · Multiple choice question · Transfer learning · OMRNet

✉ Samir Malakar
malakarsamir@gmail.com

Extended author information available on the last page of the article

1 Introduction

Multiple Choice Question (MCQ) based assessments are widely used as this evaluation system is fast and reliable. It removes subjectivity, reduces inter-evaluator variation/bias and only tests the knowledge of the candidates.

In general, any automated MCQ-based assessment system follows the four steps below:

1. **Digitizing the answer sheet** - This is usually performed with the help of scanners. With advancements in digital camera technology and efforts to reduce costs for this operation, nowadays, answer sheets are being scanned using cameras on webcams and mobiles [20, 31, 48].
2. **Candidates' information and answer boxes extraction** - For this, a specific template, usually a digital answer sheet, is used. The required information and answer boxes are usually extracted using image registration methods.
3. **Acquiring candidate information** - This process is used to recognize candidates' identification information, which is beyond the scope of this work. However, it is performed by either detecting a particular pattern on the answer sheet used in the OMR system as described in [20] or using advanced Optical Character Recognition (OCR) techniques as demonstrated by Chai [11].
4. **Recognition of responses** - Usually, for grading, an answer box is classified into two categories: marked and unmarked (empty). Therefore, simple image processing techniques can be applied to segregate a box based on a specific threshold of ink present on the box. A vastly empty box would refer to an unmarked answer, whereas a vastly filled box would qualify as a marked box.

Step 4 among the above steps limits the flexibility of traditional MCQ based assessment processes as it doesn't provide any scope for crossing out an erroneously marked answer. Figure 1 illustrates the problem in greater detail. Furthermore, any unintentional mark or scratch on the answer sheet (see Fig. 1) may cause standard OMR systems to misrecognize answers. Furthermore, a small misalignment of the scanner with respect to the OMR sheet adversely affects the ability of algorithms to classify answer boxes [5, 50].

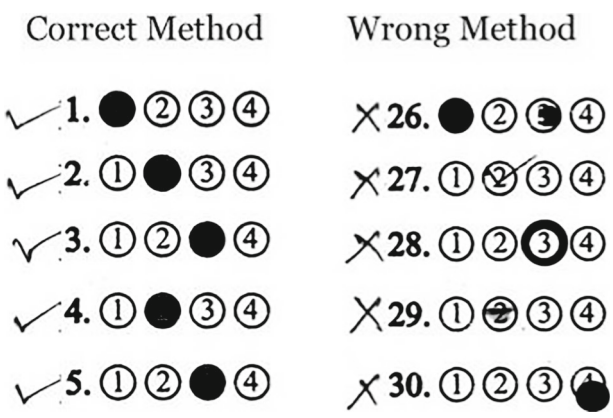


Fig. 1 A typical set of correct (responses 1-5) and incorrect (responses 26-30) protocols for marking responses in an answer sheet is used in OMR systems. These protocols do not allow for crossing out of an answer. Even if a bubble slightly deviates from its standard marking process (responses 26 and 30), the OMR machine may fail to recognize the correct answer

Generally, answer sheet templates vary depending on the assessment systems. This forces both the candidate and the evaluating team to follow a specific set of instructions to smoothly prepare and utilize the answer sheet template used in the OMR system. Consequently, misprints and unplanned deviations from the reference sheet often lead to erroneous evaluations.

To overcome these constraints, we need a flexible OMR system that provides an opportunity to cancel out a previously marked option and accommodate small misalignments. Thus, this type of OMR system needs to classify an answer box into three classes: confirmed, crossed out, and empty. A sample answer sheet containing such markings is shown in Fig. 2.

A brief overview of the 3 types of answer boxes that might appear in an answer sheet supported by the flexible OMR system mentioned above is described below, while some sample answer box images are shown in Fig. 4.

– Confirmed - The answer boxes that are chosen by the candidates as the correct answer. The correct answer can be marked by filling the answer box (see Fig. 3) or by putting a tick mark on a answer (see Fig. 2).

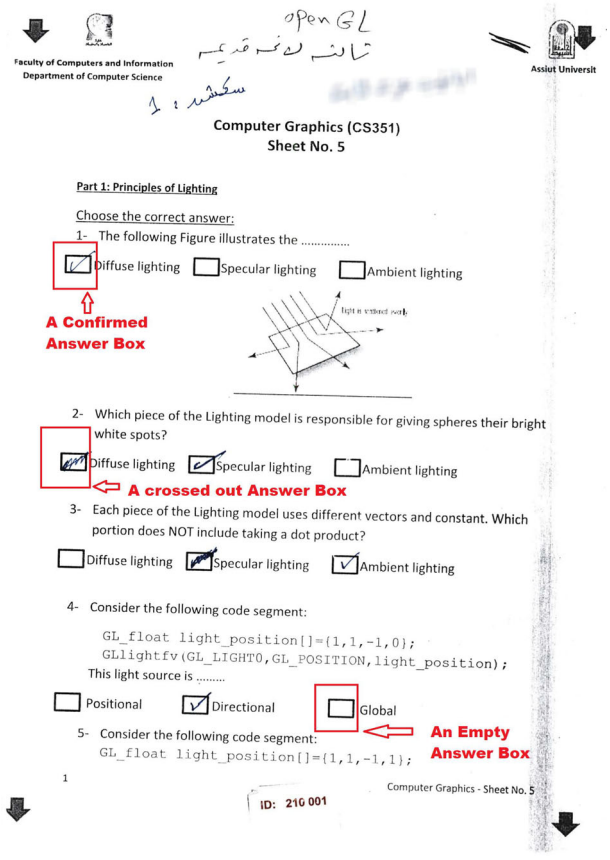


Fig. 2 A sample answer sheet used in flexible OMR systems where confirmed, crossed out, and empty answer boxes are marked in red coloured rectangular boxes

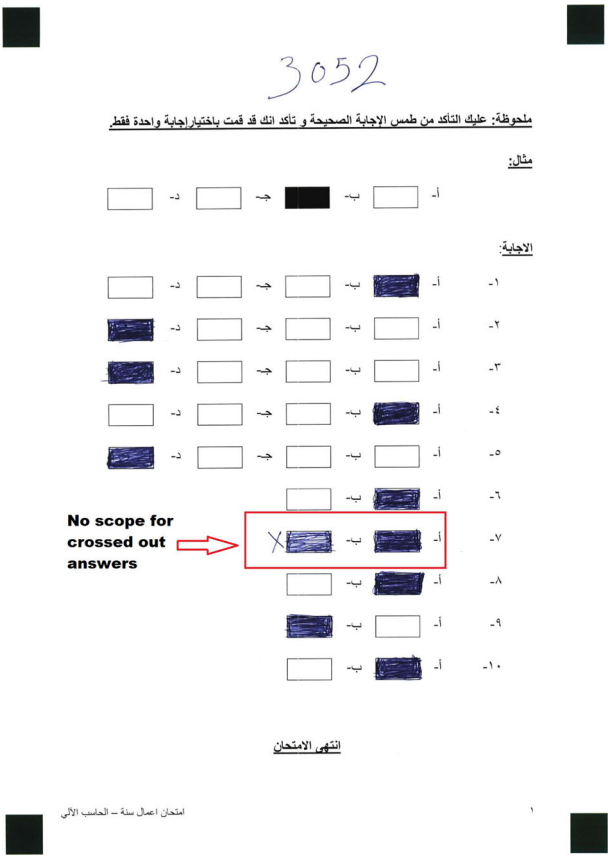


Fig. 3 A sample answer sheet with confirmed, crossed out and empty answer boxes. Here the answer box corresponding to the confirmed answer is filled fully

– Crossed out - Candidates can cancel their responses by crossing out the previous markings. Two different methods for crossing out previous responses are shown in Figs. 2 and 3.
– Empty - The blank answer box. Sometimes they may contain some dot marks or noise components.

The answer box samples shown in Fig. 4 and the above-mentioned variations that might occur indicate the complexity of the problem. Especially, distinguishing confirmed and crossed-out responses (see Fig. 3) can be quite challenging. These samples have been taken from the open access dataset known as Multiple Choice (MC) Answer Boxes dataset[1].

Apart from the mentioned challenges while designing a flexible OMR system, such a system has to be prepared for mobile and edge devices such as scanners, which tend to be expensive and require additional hardware or software support. Hence, an OMR system in the form of a mobile app is the most desirable in the present technology-enabled society. However, the processing power and computational complexity of such devices are scarce.

[1] https://sites.google.com/view/mcq-dataset

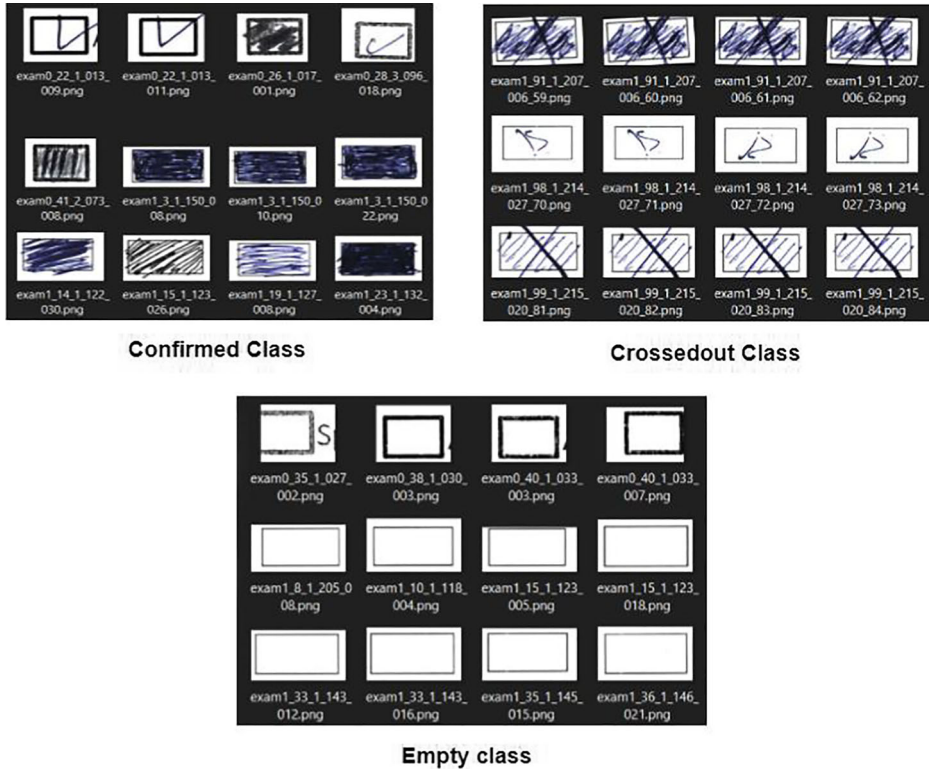


Fig. 4 Some samples of the answer box images used. The samples are taken from the MC Answer Boxes dataset

Thus in deep learning research in modern days, we can find significant developments for making a lightweight model, as can be found in the works [6, 45].

In light of the above discussions, in this work, we have tried to come up with a lightweight neural network architecture that can easily classify test answer box images into any one of the three classes: confirmed, crossed out, and empty. We have named our model OMRNet. It consists of a MobileNet V2 [49] backbone followed by 4 fully connected layers each having a dropout of 0.25 and a final softmax layer. In this work, instead of training a model from scratch, we have used transfer learning, which requires fewer training samples to train a model properly. OMRNet thus uses a 4-layer fully connected (FC) network on top of the image feature maps obtained from pre-trained MobileNetV2 (trained on the ImageNet dataset). Furthermore, we have used a post-training quantization technique to quantize the proposed model and reduce its size before deployment.

The major contributions of our work are summarized below -

- We have designed a lightweight architecture that can classify an answer box into one of three classes: confirmed, crossed out, and empty.
- We have used the concept of transfer learning as it helps train a model with fewer training samples.
- Two relatively new non-monotonic activation functions: Power Function Linear Unit (PFLU) and Faster PFLU (FPFLU), introduced by Zhu et al. [63], have been used in conjunction with Rectified Linear Unit (ReLU) [3] in the model layers.
- The entire architecture is evaluated on a publicly available dataset. Experimental results confirm the superiority of OMRNet over state-of-the-art methods.
- We have deployed our proposed classifier to a web app using a Representational State Transfer Application Programming Interface (REST API), which on uploading an answer box, returns its predicted class and confidence score.
- We have used a quantization algorithm to reduce the size of the trained model so that it can be used on edge devices and webapps through REST APIs. The new size is 7 times lesser as compared to the non-quantized model with a 0.40% drop in average accuracy on the present test set.

The rest of the paper is organized as follows: Section 2 describes the literature survey of the major works done in this field. Section 3 elaborates on our proposed work, where we have tried to address the justification of different steps used here. Section 4 includes a description of the dataset in use, testing and training protocols, performance analysis in terms of execution time, model size, recognition accuracy, comparison with state-of-the-art techniques and CNN models, etc. In Section 5, we conclude the paper and discuss the possible future extensions of this work.

2 Literature survey

In the past, OMR devices, in general used costly and bulky image scanners (hardware) for grading responses to any MCQ tests. Here we have discussed a few of the works that are most relevant to our work. However, readers can gain a better understanding of the problem by reading the review work by Elias et al. [19], in which the authors discuss the advancements and development of OMR technologies in greater detail.

Hussman and Deng [26] developed a fast OMR hardware that was capable of dealing with three different shapes of MC answer boxes: circle, oval, and rectangle. Some of the early OMR systems used a histogram of binary image pixels to recognize the MC answer boxes

[14]. However, with the increasing popularity of MCQ based assessments, efforts to replace image scanners with edge devices like mobile phones, cameras, and webcams have started gaining popularity. Some of the notable methods in this field were proposed by Nguyen et al. [46], Rasiq et al. [48] and China et al. [13]. Nguyen et al. [46] presented a low cost system based on a digital camera instead of scanners for image acquisition and answer box extraction. They first employed skew adjustment and normalization steps to locate the answer boxes in an answer sheet used for OMR systems, and then with the help of black pixel count, they localized the answer boxes. An application for mobile based MCQ test analysis and evaluation was proposed by Rasiq et al. [48]. In this work, the researchers first scanned the MCQ answer sheet using mobile and then the answer region was cropped manually. Finally, using black pixel count, the confirmed responses were identified from the cropped region. An android application for the recognition and grading of multiple choice answer sheets was proposed by both China et al. [13] and Krisnadi et al. [36]. All these methods did not provide any option to cancel out an erroneously marked response on the answer sheet.

Other than not providing an option to cross out an answer, another major challenge that modern-day OMR systems face is the extraction of skewed region of interests (ROI) due to misalignment of scanners. Sanguanst et al. [50] showed how misalignment adversely affects the recognition success of software based algorithms. AL-Marakeby [5] showed that the majority of errors caused by modern-day thresholding algorithms are a consequence of failure to detect the marks. For OMR systems utilizing algorithms that use a reference sheet for evaluation [48], misalignment skews the results by a huge margin. To tackle this issue Chai [11] came up with a solution using a solid black mark around answer boxes. But using such a method makes a system rigid on the template used.

Another major reason for the inaccuracy of modern day OMR systems is that users are often unfamiliar with the OMR evaluation technique specific to the template they are using. Weaver and Chalkley [61] showed that having an expert for explaining to the users about the marking procedure is essential for good accuracy. However, this creates an extra overhead. Further, Khan et al. [32] showed that having rules to fill an OMR sheet leads to time waste and wrong marking.

Apart from these, some researchers opted for designing a relatively flexible OMR system where a candidate can cancel out some marked responses in the answer sheet of the underlying OMR system. Spadaccini et al. [56] introduced a new term known as squareness (defined as the absolute difference between the number of black pixels in the columns and rows), which helped in distinguishing crossed out answer boxes from the confirmed cases. Fisteus et al. [20] proposed a webcam system known as “Eyegrade", which first used a two-stage thresholding process to identify answer boxes and then followed a similar approach as described in [56] to distinguish crossed out and confirm answer boxes in the answer sheet.

Some of the recent methods have applied different image processing algorithms to improve the precision of OMR answer matching [29] in real time [28]. However, they only consider empty and confirmed cases. Sinchai et al. recently suggested an arithmetic sequence to estimate undetected existing circle choice locations, thereby improving the precision of various OMR schemes [54]. Image processing techniques have also been used to retrieve information from damaged OMR sheets [4].

Generally, a detector extracts the answer boxes from an answer sheet following one of the two approaches described below [30].

– Pixel Based - In this approach, the detector tries to identify answer boxes along with the class information by studying the pixel signatures of the image.

– Object Based - In this approach, the detector identifies answer boxes along with class information using global features formed by the coalition of the pixels (e.g., shapes on any images).

Both of the aforementioned approaches can be either supervised or unsupervised [30]. In the case of supervised learning, the classifiers try to learn the features from the answer box image samples, where it is considered that the pixels are already mapped to form the answer box images. Whereas, in unsupervised learning, the classifiers aim at grouping the pixels based on similarities and try to group them into different classes [30]. Based on these approaches, Gyamfi and Missah [22] came up with a pixel based unsupervised classification approach, wherein they proposed an OMR system to detect shaded options of candidates in MCQ type examinations. Their classification was based on the pixels making up the image, along with other image preprocessing strategies.

As evident from the above discussion, most of the work done on OMR is related to the extraction and recognition of answer boxes from answer sheets used in the OMR system. The application of CNN to classify answer boxes is fairly new and highly unexplored. This may be due to the unavailability of the voluminous data required for CNN model training.

However, a notable piece of work was carried out recently by Afifi and Hussain [1], where an ML based technique was introduced to increase the flexibility of the MCQ based test assessment process. Their major contribution was to introduce a dataset of answer boxes and to design a classifier based on their database. The proposed work is an improvement on the work proposed by Afifi and Hussain [1].

Apart from this, there are some works related to recognizing handwritten MCQ exams (where a candidate's response is either 'a', 'b', 'c' or any other character of the English alphabet). Shaikh et al. [52] proposed a system that used a personal computer, a portable scanner, and an application program that automatically corrected the handwritten answer sheets using a CNN model. Supic et al. proposed a rather simple and effective algorithm that enabled the automated reading of a limited set of handwritten answers and minimized the need for human intervention in the scanning process [57].

Some methods have also been developed to automate the process of OMR/OCR and MCQ test grading using artificial vision [9], auto resonance networks [47] and character recognition techniques [35]. Mobile and web-based applications for the evaluation of MCQ tests have also gained interest in recent years, as seen from the research by Calado et al. [10] and Rasiq et al. [48]. However, after extracting the Region of Interest (ROI) they use thresholding based algorithms to recognize the answer mark.

From the above discussion, we observe that Afifi and Hussain [1] proposed a flexible OMR system where the candidates have the option to nullify his/her responses by crossing them out. However, they suggested a two-stage CNN based method, which is computationally intensive. Also, the performance of their work is not satisfactory. Keeping these issues in mind, in this paper, our primary objective is to explore a CNN based model to improve the existing solution in terms of computation and storage requirements, and to make it compatible with edge devices. There are some associated challenges with the existing methods that we have tried to address in the proposed work. The challenges and our solutions are listed below:

– In general, methods following a two-stage or two-pass approach (e.g., [40, 41]) are computationally expensive compared to a single pass method [7]. Thus for edge devices, two-stage methods might not be suitable. Hence, we have tried to find a solution that follows a single pass approach..

– In two stage classification approach, once an object gets misclassified, it remains misclassified throughout the process [8, 17]. In many cases, a relatively weaker classifier is used at the first stage, so the chances of such misclassification remain high. To this end, a single stage solution can overcome this.
– Use of a two-stage CNN method in [1] makes the classification process not only computationally expensive but also increases the storage requirement during deployment, which should be avoided. Hence, we have explored some lightweight CNN models to address this issue.
– Finally, we have tried to make the trained model lightweight so that it can be deployed on edge devices with ease.

3 Proposed work

With the exponential increase in computing power and access to abundant data, neural networks (deep neural networks in particular) have certainly revolutionized the field of Artificial Intelligence. Hence, many complex tasks, like image recognition and segmentation across various domains, can be performed with impressive accuracy. However, this excellent accuracy of state-of-the-art models often comes at the cost of high computing power, which is far beyond the capabilities of most modern mobile devices. The problem we have tried to address (i.e., the classification of answer boxes in an OMR sheet) will find its major application in resource constrained environments. To be specific, in this work, we have proposed a lightweight model while significantly improving its accuracy on the diverse MC Answer Boxes dataset Section 4.1.

3.1 Architecture of OMRNet

Since one of the primary goals of our work is to build a lightweight OMR system, we have prepared the OMRNet on top of the MobileNetV2 model specifically designed for resource constrained environments by Sandler et al. [49]. Furthermore, because CNN-based models in general require a lot of data, we have used the concept of transfer learning in OMRNet instead of building and training the model from scratch. Thus, OMRNet includes features of MobileNetV2 (pre-trained on ImageNet) with a deep multiplier of 1.0 and an input image size of 224×224 pixels[2].

The OMRNet architecture consists of 3 dense FC layers with activation functions: RELU, PFLU, and FPFLU on the top of the convolutional part of the MobileNetV2. At the end of each layer, a dropout of 25% of the total number of parameters is added to prevent the overfitting of the model during the training process. The final FC layer is followed by a 3-class classification layer with Softmax as the activation function. The entire architecture is shown in Fig. 5.

3.2 Transfer learning

One of the great things about the deep learning industry is that most of its researchers are committed to open source. This creates an unprecedented connection between state-of-the-art research and naive researchers. This allows us to base newer models on existing cutting edge

[2] https://tfhub.dev/google/tf2-preview/mobilenet_v2/feature_vector/4

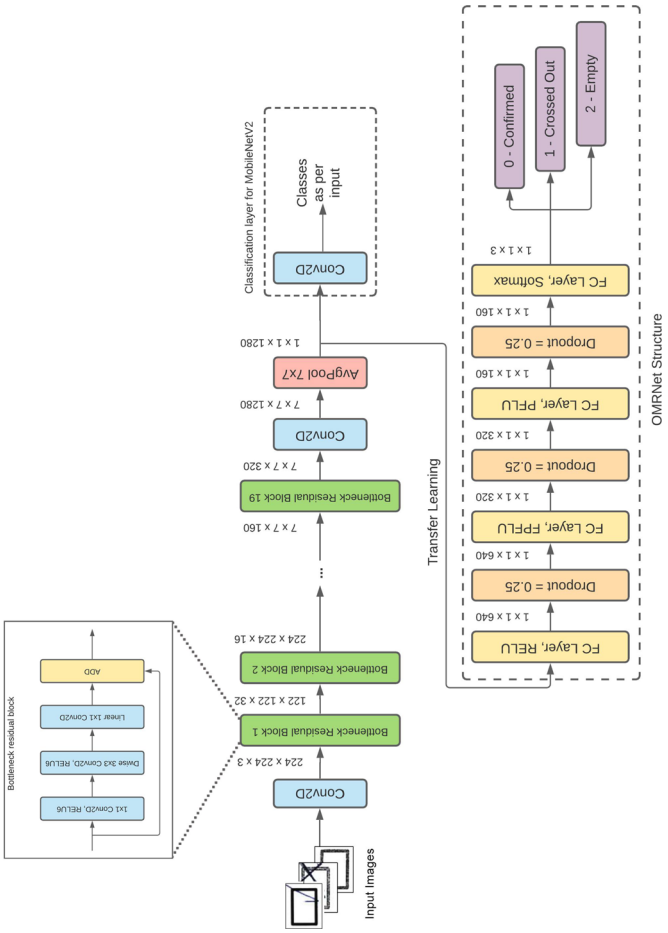


Fig. 5 Transfer Learning on MobileNetV2 [49] and our proposed model

architectures. Generally, in a CNN architecture, the filters of the convolutional layers learn the lower level features, such as edges, that are generic and are true across multiple datasets as can be seen from [6, 44]. Thus using the filters of a model trained on millions of images for a similar task can help achieve higher accuracy and reduced training times in other tasks. Since for the downstream task at hand, enough data may not be available to accurately learn those low level features, we take advantage of the features learned across a huge database.

For our purpose of answer box classification, we make use of the feature maps of the MobileNetV2 [49] model trained on the ImageNet dataset[3] [16]. Its architecture was introduced by Sander et al. [49] in 2018 with the sole purpose of improving the performance of lightweight mobile models on multiple tasks and benchmarks. This novel neural network architecture is specifically tailored for mobile and resource-constrained environments and delivers high accuracy while significantly reducing the memory footprint.

It achieves this optimal balance in performance and accuracy by keeping the parameters and mathematical operations as low as possible by using depth wise convolution. Moreover, to prevent the destruction of too much information due to nonlinearities, it introduces a new layer module: the inverted residual with a linear bottleneck. This layer encapsulates the inner layers in a bottleneck residual input[4] as shown in Fig. 5. Intuitively, it can be thought that the intermediate inputs and outputs are encoded by these linear bottleneck layers of MobileNetV2. Furthermore, it has shortcut connections between these bottlenecks to enable faster training and better accuracy. Since the major application of this answer box classifier would be on low-cost devices, we chose the image feature vectors of MobileNetV2 as our transfer learning model.

3.3 Activation functions

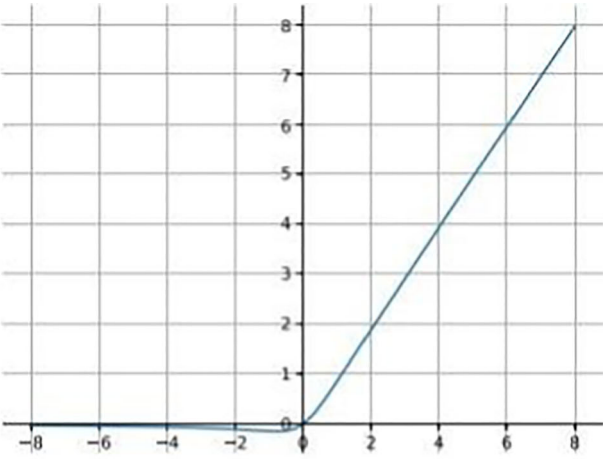
An activation function is arguably one of the most important units in a deep neural network. It is what sets a neural network apart from a logistic regression model. A neural network without an activation function would just be constrained to calculate and optimize linear functions. Traditionally, sigmoid and tanh functions are used as activation functions in the literature of deep learning. However, once the neurons get saturated, it tends to kill the gradient (as at high values, both the sigmoid and tanh functions approach zero). Since the back-propagation depends highly on the gradient this blocks the information flow through the neurons to their weights and consequently to their data[5]. To solve this issue, the ReLU activation function was introduced in [3]. In most cases, ReLU does a good job as an activation function and provides faster convergence. However, due to their fragile nature, ReLU units die irreversibly during the training process. ReLU units can get knocked off the data manifold.

To overcome this problem, in the subsequent layers, we have used two newly developed non-monotonic activation functions, namely PFLU and FPFLU, introduced by Zhu et al. [63]. The negative parts of these activation functions are non-monotonic (see Fig. 6). Therefore, these activation functions maintain the sparsity of the negative part while introducing negative activation values and nonzero derivative values for the negative part. This leads to better training than ReLU. The shapes of said two activation functions are shown in Fig. 6. Details of these activation functions are provided below.

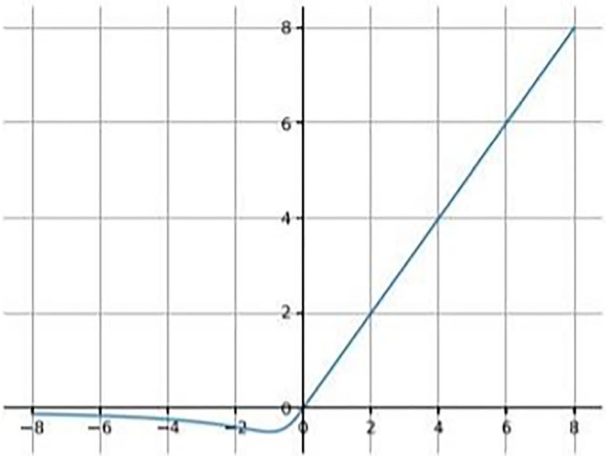
[3] https://www.image-net.org/index.php

[4] https://cutt.ly/X22ozlv

[5] https://cs231n.github.io/neural-networks-1/



(a) PFLU



(b) FPFLU

Fig. 6 Shapes of PFLU and FPFLU

1. PFLU: The function is defined in (1):

$$f(x) = x \cdot \frac{1}{2}\left(1 + \frac{x}{\sqrt{1+x^2}}\right) \tag{1}$$

Some of the important properties of PFLU are as follows:

- $f(x)$ is derivable $\forall x \in \mathbf{R}$ ($\mathbf{R}$ refers to the set of real numbers)
- $f(0) = 0$ and $f'(0) = \frac{1}{2}$
- When $x > 0,\ 0 < f(x) < x$ and $f'(x) > 0$. As $x \to \infty,\ f(x) \to x$

2. FPFLU: It is defined in (2):

$$f(x) = \max\left(x, \frac{x}{1+x^2}\right) = x \cdot \begin{cases} 1 & \text{if } x > 0 \\ \frac{1}{1+x^2} & \text{if } x \leq 0 \end{cases} \tag{2}$$

Some of the important properties of FPFLU are as follows:

- $f(x)$ is derivable $\forall x \in \mathbf{R}$
- When $x > 0$, $f(x) > 0$ and $f'(x) = 1$
- When $x \leq 0$, $f(x) \leq 0$ and $f'(x) > 0$.
  For $x \in [-1, 0]$, $f'(x) \geq 0$
  For $x < -1$, $f'(x) < 0$
  As $x \to -\infty$, $f(x) \to 0$

3.4 Fully connected layers

FC layers are defined as the layers where all the inputs from one layer are connected to every activation unit of the next layer. FC layers are generally used at the end of any neural network architecture for detecting specific global configurations of the features detected by the lower layers in the neural network (in our case, the image feature vectors of MobileNetV2). They are good at extracting high level features specific to a dataset. Thus, we have used 3 FC layers to extract the global features relevant to our answer box classification problem based on the lower level features obtained from the feature maps of the MobileNetV2 model trained on ImageNet. The FC layers follow a converging structure based on the number of features available in the MobileNetV2 feature vectors (which is 1240). Thus, the 3 FC layers have a size of 640, 320, and 160 respectively.

3.5 Dropout

Neural networks sometimes learn statistical noise as a feature of the dataset. Although this can improve accuracy in the training data, it can adversely affect test accuracy. To minimize the effect of this overfitting, we have randomly discarded 25% of neurons from each FC layer.

4 Experimental results and analysis

This section explains and analyses the experimental results obtained using the proposed OMRNet for classifying the answer boxes present in the OMR sheets. All the experiments are carried out on the free-to-use Kaggle kernel. Kaggle Kernel is a free platform that lets us run Jupyter notebooks inside a web browser itself. Further, it provides the processing power for the notebook that comes from servers in the cloud. Hence the implementation processes including the training are performed on a remote machine having Tesla P100-PCIE-16GB GPU, 2.30HZ Intel(R) Xeon(R) 4 core 13 GB CPU. We have used the Tensorflow library (version 2.4.1) to implement our model. In this section, we have described the dataset in use, the experimental setup, the OMRNet training process, and model performances by varying the different units of the OMRNet.

4.1 Dataset description

For experimental purposes, we have used the 5-fold cross-validation sets for the classification of the samples provided under MC Answer Box dataset[6]. The dataset is pre-organized for 5-fold cross validation experiments by Afifi and Hussain [1]. The dataset consists of rectangular shaped MC answer box images divided uniformly into three classes namely, confirmed, crossed out, and empty. The confirmed class consists of the answer box images which the candidates had chosen to be the correct answer, the crossed out class comprises the answer box images that were canceled by the candidates, and the empty class includes the blank answer box images.

The major reasons for choosing this dataset for our experiments are :

- There is no uniformity in the shape and size of the answer boxes. Furthermore, the images have varying light intensities. This helped us simulate the situations where the answer sheet is placed on the scanner in a skewed manner.
- This has helped us simulate situations where the scanner is skewed with respect to the answer sheet.
- The dataset contains answer sheets following various templates. This has assisted us in making our model template agnostic.
- The answer marking pattern also varies from one sheet to another. This has helped our model accommodate the varying fill patterns of various sheets.

Overall, the dataset is quite diverse, which has helped our model accommodate diverse features.

The entire dataset consists of 10200 MC Answer Box images, divided into 5 folds. Each fold contains 2040 images (680 samples per class). It is to be mentioned that to increase the number of crossed out images and balance the number of images in all 3 classes, three types of image augmentation: translation, rotation, and flipping (both vertical and horizontal) were performed by the researchers who made this dataset public. Some sample images of the dataset from fold 1 are shown in Fig. 4.

4.2 Data preprocessing

It is well understood that any ML model is only as good as the data it receives. Hence, we performed some preprocessing on the input answer box images prior to feeding the same to the OMRNet model. At the outset, every answer box image is resized to a fixed dimension of $224 \times 224 \times 3$, similar to the standard image dimension that a MobileNetV2 model takes as input. For resizing the images, we have used bi-linear interpolation, wherein a new pixel value is interpolated from a linear combination of four of the nearest pixel values [62]. Further, we have normalized each image pixel values within the range [0, 1] by dividing each pixel intensity by 255 since deep neural networks tend to work better with smaller inputs.

4.3 Training OMRNet

We have trained our OMRNet model on the training samples present in each of the 5 folds, as mentioned above. During training, we have randomly selected 20% of the training samples as validation samples, i.e., the model uses 6528 and 1632 answer box images as training and

[6] https://sites.google.com/view/mcq-dataset

validation samples, respectively. It has already been mentioned that the OMRNet uses the concept of transfer learning. Hence, during training, we have used the pre-trained MobileNetV2 weights i.e., in the OMRNet model, MobileNetV2 is used as a feature map generator. The generated feature maps are then flattened and input to the classifier designed for answer box image classification. The architecture of the classifier is shown in Fig. 5.

The parameters used when training the model are as follows:

- Batch size = 32
- Number of epochs = 10
- Loss function - Since each sample (answer box) belongs exactly to one class (either confirmed, crossed out, or empty), we have used the probabilistic loss function known as sparse categorical cross-entropy. We have evaluated the loss function based on training accuracy.
- Optimizer - We have used the Adam optimizer [33] with the following parameters.
  1. Learning_rate = 0.001
  2. The exponential decay rate for the first moment = 0.9
  3. The exponential decay rate for the second moment= 0.999
  4. epsilon = 1e-07 (a small constant value used for numerical stability [33])

The OMRNet model is trained using the above mentioned parameter values. The graphs for the training vs. validation accuracy and the sparse categorical cross entropy loss of the model with varying epochs for the fold 1 experiment are shown in Figs. 7 and 8 respectively. These graphs are evidence that the training process is convergent. It is to be noted here that the training results for the other folds also converged within 10 epochs. However, we have only shown the graphs for the Fold1 experiment.

4.4 Performance on test data

After the completion of model training, we evaluated the performance of the trained module on unseen test samples (i.e., the answer box images of the reserved fold for testing). The recognition accuracy obtained using the 5-fold cross validation scheme is shown in Table 1. From the results, it is observed that the best recognition accuracy is obtained for the fold 4

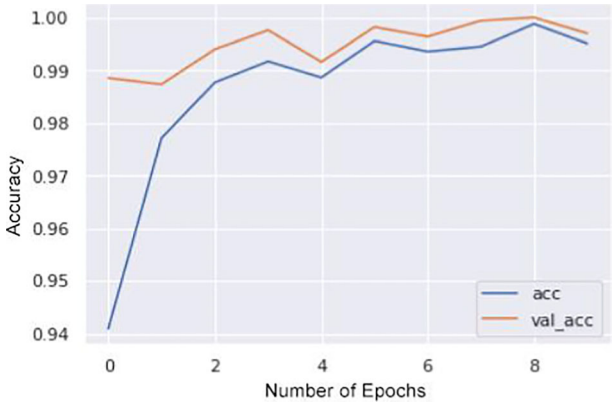


Fig. 7 Plot representing training and validation accuracies vs the number of epochs using the proposed OMRNet

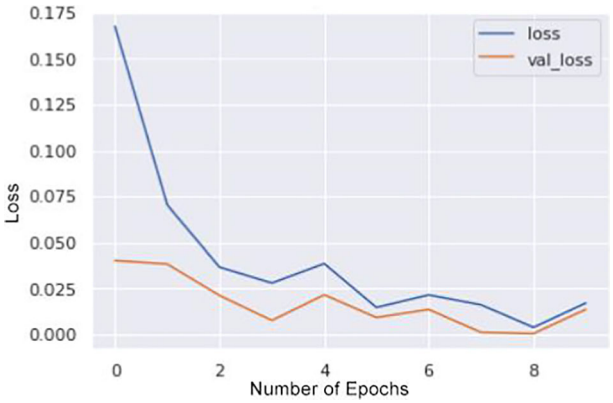


Fig. 8 Plot representing training and validation losses vs the number of epochs using the proposed OMRNet. The loss is calculated using a sparse categorical cross-entropy loss function

Table 1 Test accuracies (in %) of OMRNet obtained by using the 5-fold cross validation scheme

Model	Fold1	Fold2	Fold3	Fold4	Fold5	Avg.
OMRNet	95.29	95.88	93.97	97.45	97.20	95.96

experiment, which is 97.45% while in the fold 3 experiment, OMRNet performs the worst and achieves 93.97% recognition accuracy. OMRNet recognizes the test answer box images with 95.96% average accuracy. Some correctly recognized answer box images are shown in Fig. 9.

4.5 Comparative analysis

Although the proposed OMRNet performs well, we have performed some additional experiments while varying pre-trained CNN models, activation functions, optimizers, dropout rate, and hidden layer size to test its effectiveness for solving the problem at hand. All these experiments have been conducted using the 5-fold cross validation scheme. For each new experiment, the models are retrained using the same training process as described in Section 4.3.

4.5.1 Accuracy comparison with respect to state-of-the-art CNN architectures

To see how our model performs against other state-of-the-art models, we have trained those models on the dataset we have used. However, our proposed OMRNet outclassed all its competitors by some distance. Among them, EfficientNet-B0 [59] had the highest average accuracy of 94.05%. The actual MobileNetV2 as proposed by Sanders et al. in the paper [49] gave the second highest accuracy of 94.05%. All other models (DenseNet-201 [25], VGG-19 [53] & ResNet-50 [23]) performed poorly and couldn't even attain 85 % average accuracy. In particular, ResNet-50 performed significantly poorly and gave an average accuracy of 55.48 %. Our results are summarized in Table 2.

We have further performed the Wilcoxon rank-sum test to check whether the performances obtained over different folds are statistically significant or not. We have conducted this test

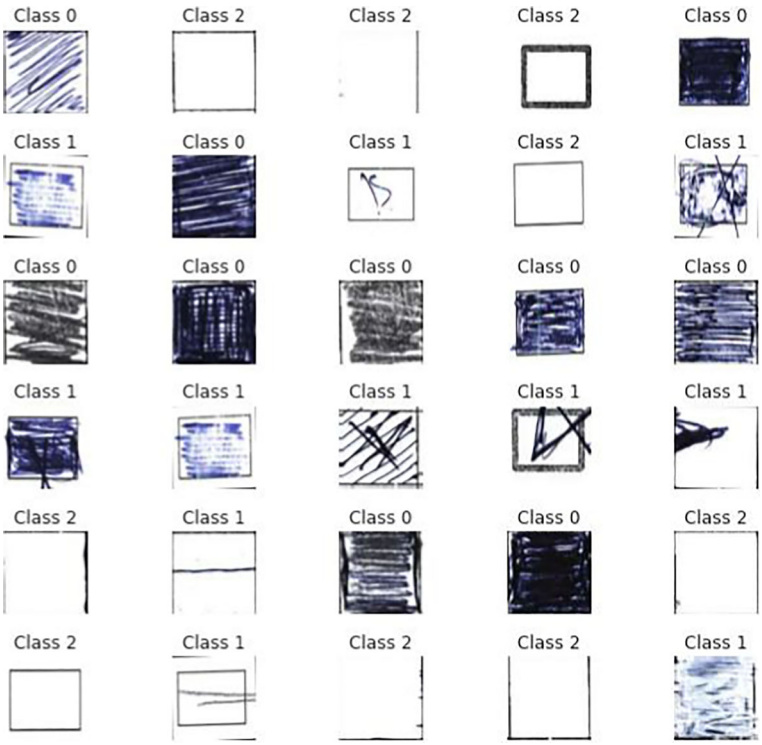


Fig. 9 Some correctly recognized answer box images. Here 0, 1, and 2 represent confirmed, crossed out, and empty answer boxes respectively

following the process mentioned in the works [18, 39, 43]. For this, we have set the null hypothesis: the performance of OMRNet is not significantly different from the performance of the other CNN models mentioned in Table 2. We can reject the null hypothesis at $a\%$ confidence level if all the estimated p-values are less than $a/100$. Based on the performances mentioned in Table 2, we have estimated the p-value, and the estimated $p-$values are recorded in Table 3. From the $p-$values recorded in Table 3, we can reject the null hypothesis at an 11% confidence level. Therefore, we can safely claim that the performance of the OMRNet is significantly better than the performance of other CNN models listed in Table 2. If we closely analyze the estimated $p-$values then we can observe that the performance of OMRNet is not significantly different from EfficientNet-B0 at a 5% confidence level, because this CNN model has provided very close results as compared to OMRNet.

4.5.2 Comparison of test accuracies while varying pre-trained CNN models coupled with proposed modifications

For developing OMRNet, we have used transfer learning on the feature vectors of pre-trained MobileNetV2. However, we have also tried the feature vectors of other popular pre-trained models to select the best fit for our purpose. We have compared the results of our architecture by changing the transfer learning model from MobileNetV2 to other state-of-the-art CNN models that include VGG19 [53], DenseNet201 [25], EfficientNet-B0 [59], Efficient-B1

[59], Inception-V4 [58], and MobileNet-V3 [24]. The comparative results have been shown in Table 4. The results clearly establish that use of the feature vectors of MobileNetV2 in OMRNet as a pre-trained CNN model is justified since it performs the best among the CNN models used here for comparison. Here also, we have performed statistical significance tests as we have done in Section 4.5.1. and the $p-$values recorded in Table 5. Here also, the use of MobileNetV2 proved to be statistically significant at 20% confidence level. $p-$values in Table 5 prove that the performance of EfficientNet-B0 is closely related to the performance of MobileNetV2 for this set of experiments.

Table 2 Comparison in terms test accuracy (in %) of proposed OMRNet with other state-of-the-art CNN models. The bold-face data represent the best performance (average case) and number of parameters

Model	Fold1	Fold2	Fold3	Fold4	Fold5	Avg.	Parameters
DenseNet-201 [25]	85.39	80.74	80.44	83.92	84.90	83.08	21.1 M
VGG-19 [53]	79.20	77.30	78.48	82.16	82.79	79.99	144.5 M
MobileNetV2 [49]	92.30	95.15	92.21	95.44	95.15	94.05	**4.4 M**
EfficientNet-B0 [59]	94.36	94.66	92.06	95.83	95.49	94.48	6.2 M
MobileNetV3 [24]	56.67	57.71	53.12	61.62	59.93	57.81	6.4 M
EfficientNet-B1 [59]	33.33	34.35	36.33	37.30	35.33	35.32	8.7 M
Resnet-50 [23]	53.82	53.77	54.75	54.85	60.20	55.48	26.5 M
OMRNet	95.29	95.88	93.97	97.45	97.20	**95.96**	**4.4 M**

Table 3 p-values of the Wilcoxon rank-sum test for the results recorded in Table 2

DenseNet-201	VGG-19	MobileNetV2	EfficientNet-B0	Resnet-50	MobileNetV3	EfficientNet-B1
0.0086	0.0086	0.0525	0.1076	0.0086	0.0086	0.0086

Here, $p-$ values are calculated with respect to OMRNet

Table 4 Comparison of the test accuracy (in %) of MobileNetV2 (used in our proposed model OMRNet) with state-of-the-art CNN models used in the feature extraction part following the transfer learning concept. The bold-face number represents the best performance in the average case

FV of Model	Fold1	Fold2	Fold3	Fold4	Fold5	Avg. Acc
EfficientNet-B0 [59]	95.29	92.70	94.71	96.86	96.23	95.16
VGG-19 [53]	89.22	87.30	88.48	92.16	92.79	89.99
DenseNet-201 [25]	87.50	87.25	86.96	94.17	91.32	89.44
ResNet-50 [23]	87.30	84.56	81.72	76.13	86.96	83.33
EfficientNet-B1 [59]	92.45	94.85	93.92	96.18	96.96	94.87
Inception-V4 [58]	93.19	94.41	87.99	93.63	95.44	92.93
MobileNetV3 [24]	93.97	96.03	92.79	94.65	95.93	94.67
[6pt] MobileNetV2	95.29	95.88	93.97	97.45	97.20	**95.96**

FV refers to Feature Vectors of the models

Table 5 p-values of the Wilcoxon rank-sum test for the results recorded in Table 4

EfficientNet-B0	VGG-19	DenseNet-201	ResNet-50	Efficient-B1	Inception V4	MobileNet V3
0.1954	0.0086	0.01415	0.0086	0.1471	0.0350	0.1471

Here, $p-$ values are calculated with respect to MobileNetV2

4.5.3 Accuracy of OMRNet with varying activation functions

OMRNet uses ReLU, FPFLU, and PFLU, respectively, as the activation functions of its 3 FC layers. We have evaluated the proposed model performances by changing the activation functions used in the OMRNet to other popular activation functions. Initially, we compared its accuracy by keeping the same activation function for all three layers. We used ReLU, FPFLU, FPFLU, Leaky ReLU, Scaled Exponential Linear Unit (SELU), Exponential Linear Unit (ELU), tanh, and Parametric Rectified Linear Unit (PReLU) which are shown in Table 6. However, none of them could beat the accuracy obtained by using a conjunction of ReLU, FPFLU, and PFLU for the three FC layers, respectively. Our conjunction once again outperformed its contemporaries by some margin. Leaky ReLU with an alpha value of 0.01 had the highest average accuracy among them at 95.06%. Here, alpha is the small gradient when the unit is not active.

Since Leaky ReLU (with an alpha value of 0.01) gave the highest accuracy, we tried replacing it with ReLU as the activation of our first layer. But it gave an average accuracy 95.04% as compared to our model's accuracy of 95.96% further justifying our approach to use the ReLU, FPFLU, and PFLU conjunction. We have also compared the order of the activation functions with 6 ($3 \times 2 \times 1$) of the possible permutations in their position, and there too it stands out from the rest as depicted in Fig. 10.

Table 6 Comparison of test accuracies (in %) with varying activation functions. The bold-face number represents the best performance in the average case

Activation Functions	Fold1	Fold2	Fold3	Fold4	Fold5	Avg.
SELU	92.55	94.26	91.91	96.32	95.93	94.20
ELU	93.38	95.10	93.09	93.19	94.75	93.90
tanh	95.74	90.20	92.65	94.90	95.00	93.70
PReLU	92.94	95.20	93.38	93.58	96.13	94.25
FPFLU	91.22	94.07	92.79	97.30	96.13	94.30
PFLU	94.46	94.22	91.27	94.75	95.29	94.00
LReLU (0.1)*	93.48	92.94	89.02	94.90	96.22	93.31
LReLU (0.01)*	94.56	93.73	93.92	98.24	94.85	95.06
ReLU	93.68	93.92	94.17	95.83	96.57	94.83
[6pt] Our Conjunction**	95.29	95.88	93.97	97.45	97.20	**95.96**

*The value in the bracket beside LReLU refers to the alpha value, which is the small gradient Leaky ReLU provides when the unit is not active

**Our Conjunction refers to the order of activation functions used in the proposed OMRNet, using ReLU for the first layer, FPFLU for the second layer, and PFLU for the third layer

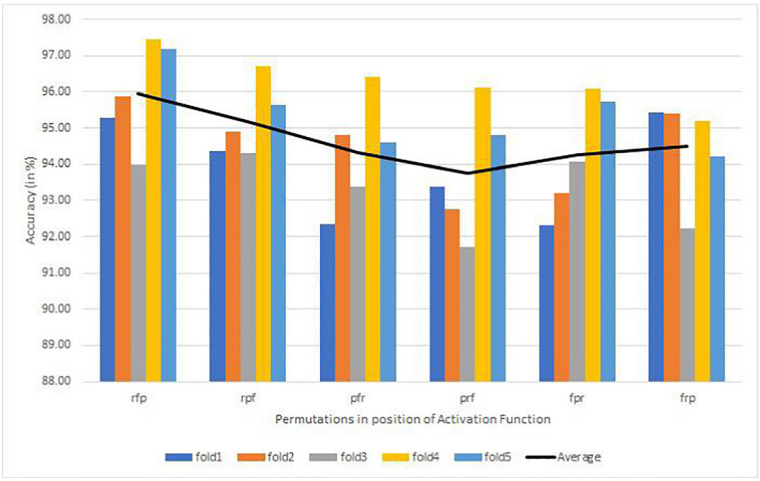


Fig. 10 Variation of test accuracies with respect to permutations in the position of the activation function in the FC layers we have used. Here, r, p and, f represent ReLU, PFPLU, and FPFLU activation functions, respectively

4.5.4 Accuracy of OMRNet with varying optimizer

We have used Adam as an optimizer with parameters as described in Section 4.3. Comparison with other state-of-the-art optimizers like Stochastic Gradient Descent (SGD) and RMSProp has further proved the performance of our model as it outperforms both of them in terms of accuracy. None of them could guide the model to an accuracy greater than 95%. The results are elaborated in Fig. 11.

4.5.5 Accuracy of OMRNet with varying dropout rate

Dropout prevents a model from overfitting, however excessive dropout tends to make it an underfit. Thus keeping a balance is essential. We have compared our model by changing

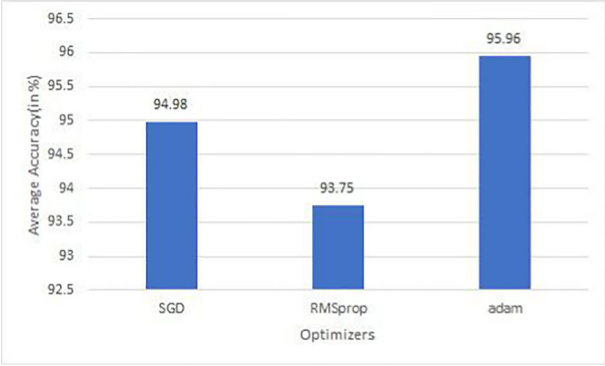


Fig. 11 Comparison of the test accuracy while using different Optimizers

Table 7 Comparison of test accuracies (in %) with varying Dropout rate in each FC layer. The bold-face number represents the best performance in the average case

Layer 1	Layer 2	Layer 3	Fold1	Fold2	Fold3	Fold4	Fold5	Avg
0.50	0.50	0.50	95.05	95.98	93.43	98.09	94.75	95.46
0.20	0.20	0.20	94.07	94.22	92.50	96.32	96.37	94.70
0.25	0.25	0.25	95.29	95.88	93.97	97.45	97.20	**95.96**
0.50	0.50	0.25	93.58	95.34	93.97	96.18	96.18	95.05
0.25	0.25	0.50	92.35	94.80	92.70	95.69	95.88	94.28
0.25	0.50	0.50	94.26	94.51	91.62	96.57	94.90	94.37
0.50	0.25	0.50	94.22	94.95	93.58	94.51	97.50	94.95
0.50	0.25	0.25	95.10	93.58	94.66	96.67	95.59	95.12

dropout across the 3 layers but here too our model gives the best results in comparison with others. The results are summarized in Table 7.

4.5.6 Accuracy of OMRNet with varying number of neurons in a layer

The presence of excessive neurons can make a neural network learns statistical noise in the dataset, whereas their deficiency can cause the model to catch lesser details of the dataset. We have used 640, 320, and 160 neurons for the 3 FC layers respectively. Our model successfully strikes an optimum balance between the two and outperforms its variants as demonstrated in Table 8.

4.5.7 Accuracy of OMRNet with other activation functions in the output layer

Although Softmax has become a clear choice for activation of the output layer in multi-class classification, we have still varied it with ReLU and Leaky ReLU. As before, this time also our model proves its mettle and emerges as clear winner as shown in Table 9.

Table 8 Comparison of test accuracies (in %) with varying number of neurons in each FC layer. The bold-face number represents the best performance in the average case

Layer 1	Layer 2	Layer 3	Fold1	Fold2	Fold3	Fold4	Fold5	Avg
512	256	128	94.36	95.00	94.22	97.21	96.47	95.45
1024	512	256	94.85	94.66	92.30	96.91	94.75	94.70
1240	640	320	94.22	93.09	94.46	96.23	95.74	94.75
640	320	160	95.29	95.88	93.97	97.45	97.20	**95.96**
640	640	640	94.51	94.41	92.99	93.82	93.58	93.86
320	320	320	94.07	94.56	92.35	97.11	94.71	94.56
640	640	320	94.85	93.19	92.79	95.83	90.00	93.33
640	320	320	93.87	93.04	92.06	95.78	94.07	93.76
320	320	160	93.53	94.17	94.41	95.88	96.52	94.90
320	160	160	91.76	95.39	93.09	93.43	95.74	93.88

Table 9 Comparison of test accuracies (in %) with varying classification activation layer. The bold-face number represents the best performance in the average case

Activation	Fold1	Fold2	Fold3	Fold4	Fold5	Avg.
SOFTMAX	95.29	95.88	93.97	97.45	97.20	**95.96**
ReLU	94.56	94.90	95.05	97.06	90.78	94.47
LRELU (0.01)	93.43	96.52	94.36	93.68	95.44	94.69

4.6 Performance analysis in terms of execution time and network size

One of the best features of our model is its lightweightness. We have been able to strike an optimum balance between accuracy and performance in terms of memory efficiency. Even though our model does significantly better than other state-of-the-art architectures (as discussed in the previous sections), it does not come at the cost of a high processing power requirement. This allows it to be a perfect fit for deploying on mobile and edge devices and other resource constrained environments. Two of the most important parameters for a model to be lightweight are its speed and size. Thus in the following subsections, we have compared our model's speed and size with state-of-the-art models.

4.6.1 Training and evaluation time comparison

Speed is a crucial ingredient for any model that has real life applications. A faster model can lead to the faster calculation of results and is often the need of the hour. Furthermore, a faster model generally implies less computing power being drawn.

In the following subsections, we have first compared our model's performance by varying feature vectors with state-of-the-art models' feature vectors (all of which are trained on ImageNet). Next, we have compared our model with the performance of various state-of-the-art models trained on the dataset we have worked on.

All these performances are evaluated on Kaggle kernels as described at begining of Section 4. For the purpose of comparing the speed, we have used two parameters namely -

1. **Total training time** - This refers to the total time the model takes to train on the five folds. A model that takes higher time to train would be undesirable.
2. **Evaluation time** - This refers to the the time the trained model takes to classify all the images in the five folds. A heavier architecture would take a longer time to classify an image. Since an image can come in any shape or size, we use the time it takes to classify all the images of the dataset to get a good estimate.

OMRNet takes 8 minutes and 28 seconds to train and it takes 22.4 seconds to classify all the answer boxes of the dataset. This implies on average it can classify 455.35 answer boxes per second (on the Kaggle kernel).

Performance comparison by using feature vectors of other models: We have used the feature vectors of MobileNetV2 [49] for building OMRNet. It has given both better performance and accuracy compared to other popular architectures (DenseNet-201, EfficientNet-B0, VGG-19, and ResNet-50). EfficientNet-B0's feature vectors had the second fastest training times, i.e., 10 minutes 22 seconds. Using the feature vectors of VGG-19 made the model the slowest, and it took 65 times more time for evaluation and 119 times more time for training. This means that on average it can classify a mere 7.07 answer boxes per second. Similarly, the results from DenseNet-201 and ResNet-50 were not up to the mark. The results are summarized in Table 10.

Comparison of OMRNet in terms of speed with state-of-the-art models: We have compared the speed of OMRNet with some other state-of-the-art models using the parameters mentioned in the Section 4.6.1. OMRNet beats the fastest among them (MobileNetV2) by 51 seconds in training and 3.9 seconds in evaluation time. Once again, VGG-19 is the slowest. EfficientNet-B0 is the second fastest, followed by ResNet-50 and DenseNet-201. The details are summarized in Table 11.

4.6.2 Performance with respect to size

A smaller size makes a model more effective and versatile. More specifically, a smaller trained model is better suited to a resource-constrained environment. Here, we trained some popular

Table 10 Training and Evaluation time comparison by using feature vectors of other models on the OMRNet model

FV of Model	Training Time	Evaluation Time
DenseNet-201	21 min 58 sec	41.9 sec
EfficientNet-B0	10 min 22 sec	26 sec
VGG-19	955 min 50 sec	1442 sec
ResNet-50	13 min 57 sec	31.9 sec
BiT-S	51 min 02 sec	65 sec
Inception-ResNet	46 min 15 sec	61 sec
MobNetV2 (OMRNet)	8 min 24 sec	22.4 sec

A feature vector is a tensor of numerical features that represent an object, where each element in the tensor corresponds to a specific feature or attribute of the object
FV refers to the feature vectors of the models

Table 11 Training Time and Evaluation Time comparison of OMRNet model with respect to other pre-trained models

Model	Training Time	Evaluation Time
DenseNet-201	21 min 9 sec	41.1 sec
EfficientNet-B0	11 min 37 sec	28 sec
VGG-19	944 min 32 sec	1398 sec
ResNet-50	13 min 46 sec	31.8 sec
MobileNetV2	9 min 11 sec	26.3 sec
OMRNet	8 min 24 sec	22.4 sec

These models were pre-trained on the ImageNet dataset (upstream pre-training) and finetuned on the MC Answer boxes dataset (downstream fine-tuning)

Table 12 Comparison of the size of our model with state-of-the-art models

Model	Size in MB
VGG-19	548
DenseNet-201	78
EfficientNet-B0	20.74
ResNet-50	98.14
OMRNet	21.1

Our model has the second most smallest size among all of them. It is just 0.36 MB larger than the smallest model

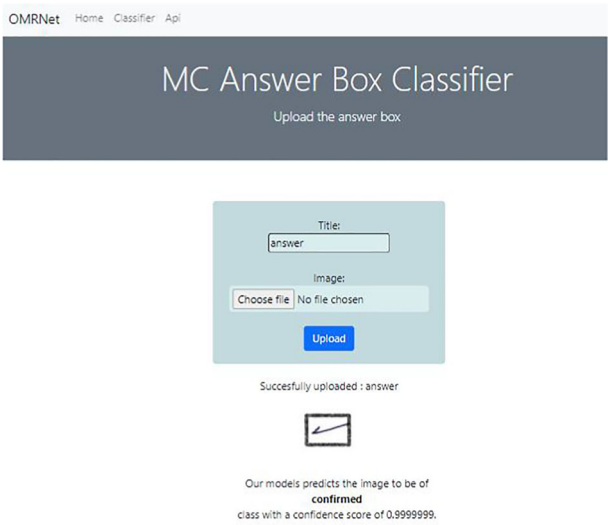


Fig. 12 An snapshot of the web app we have developed. The site allows users to upload an answer box in standard image formats (like .jpg or .png) and it predicts the class of the answer box with a certain confidence score. It uses our proposed model at the back end

CNN models like VGG-19, DenseNet-201, EfficientNet-B0, and ResNet-50) on the present training dataset and evaluated their training model's size. The trained model of OMRNet is of size 21.1 MB. VGG-19 was the largest with a size of 548 MB, followed by ResNet-50 with a size 98.14 MB, and DenseNet-201 having a size of 78 MB. EfficientNet-B0 however does slightly better than OMRNet, having a size of 20.74 MB, which is 0.36 MB smaller than OMRNet. However, since our model does significantly better in all other sections, we can safely claim our pre-training model is a better performer than Efficient-B0 and all other aforementioned models on the working dataset. The results are summarized in Table 12.

4.7 The Web App and the REST API

We have deployed the OMRNet to a web app. The webapp provides an easy to use User Interface (UI) for uploading an answer box. It then predicts the class of the uploaded answer box along with its confidence score for the classification. It does so by passing the uploaded image to OMRNet and returning back its predictions. The confidence score refers to the probability with which OMRNet claims an answer box to be to a particular class. A snapshot of the UI of the website is shown in Fig. 12.

Generally, an API is used to allow two software applications to communicate and exchange data with each other. A REST API is used to describe a set of resources and a set of operations that can be called on those resources. These operations in a REST API can be called from any Hypertext Transfer Protocol (HTTP) client, including a web app, a desktop app, or a mobile app. Hence, we have built a REST API for the classification of extracted answer boxes with a wide variety of shapes. It exposes OMRNet through an HTTP endpoint, and it lets potential users use it via the internet. The REST API on receiving a post request with an image and

title returns back a json [7] response consisting of OMRNet's prediction of the inputted answer box image with the confidence score for the made prediction.

The limitations of step 4 (recognition of responses) in any automated MCQ based assessment system, as discussed in Section 2 can be overcome by using the service of our classifier. Users who have OMR systems can upgrade their existing systems to incorporate crossed out answers by using our classifier. It will increase the flexibility of their grading schemes. Existing systems that can extract the answer boxes using orthodox ROI extraction methods (as discussed in Section 2) can send a post request to our API to know its class (confirmed, crossed out, or empty). Once the class of an answer box is known, they can score the answer sheet accordingly. A snapshot of the REST API of our model is shown in Fig. 13.

One of the main aims of developing a REST API for OMRNet is to facilitate its deployment to edge devices like smartphones. To tackle the problem of limited computational resources and storage in such devices we have used model quantization to improve the lightweight nature of our model, thus making its deployment faster and less storage intensive. We have performed post-training for our model using the TensorFlow Lite and TensorFlow Model Optimization libraries. Ultimately, the final model size before deployment through the REST API is 3 MB with a drop in accuracy of 0.40% on average.

4.7.1 Model quantization

Model quantization is a technique used to reduce the memory and computational requirements of deep learning models. It is typically performed by reducing the number of bits used to represent the weights and activation of a model. We have used post-training model quantization with the following steps.

- Training: The entire precision model has been trained on the present dataset as usual.
- Quantization: The weights and activations of the trained model have been quantized to a lower-bit representation. We have used the algorithm proposed by [27] for this purpose.
- Retraining: The quantized model has been retrained on the dataset to fine-tune the quantized weights and activation. This step has been performed to compensate for the loss of accuracy caused by quantization.
- Optimization: The quantized model has been optimized for deployment by pruning unimportant weights and removing unnecessary layers.

We have used the quantization algorithm proposed by Jacob et al. [27] which uses integer arithmetic to approximate the floating-point computations in a neural network. This helped us reduce the trained model size 7 times. From 21 MB we were able to reduce the model size to 3 MB. The results of the model quantization are summarized in Fig. 14. In this figure, the first five scores represent the performances of each fold of five-fold experiments, while the last column represents the average test accuracy. As evident from the figure, post quantization the accuracy drops by 0.4% on average.

4.8 Discussion and error analysis

The confusion matrices of the model trained on the 5 folds are shown in Figs. 15 and 16), where 0, 1 and 2 represent confirmed, crossed out and empty classes, respectively.

[7] https://www.json.org/json-en.html

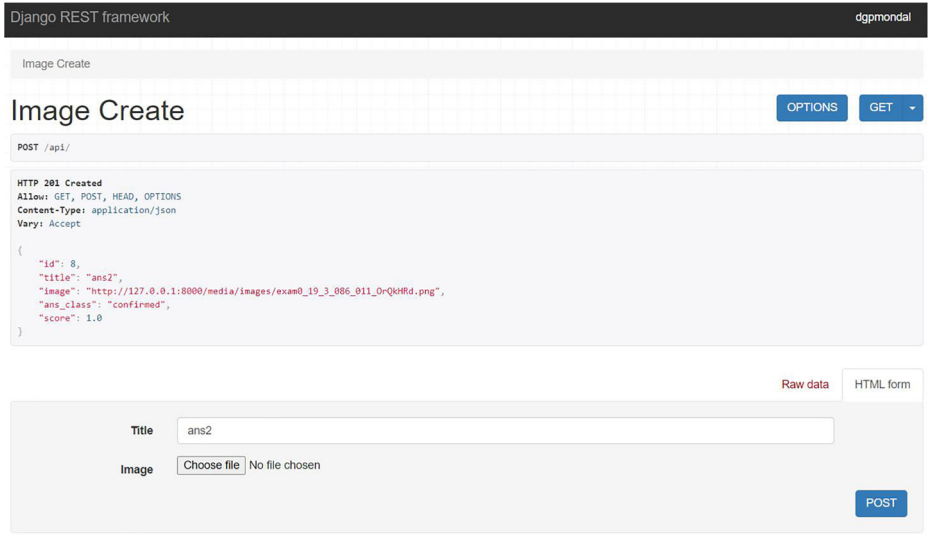


Fig. 13 A snapshot of the REST API view for answer box classification app

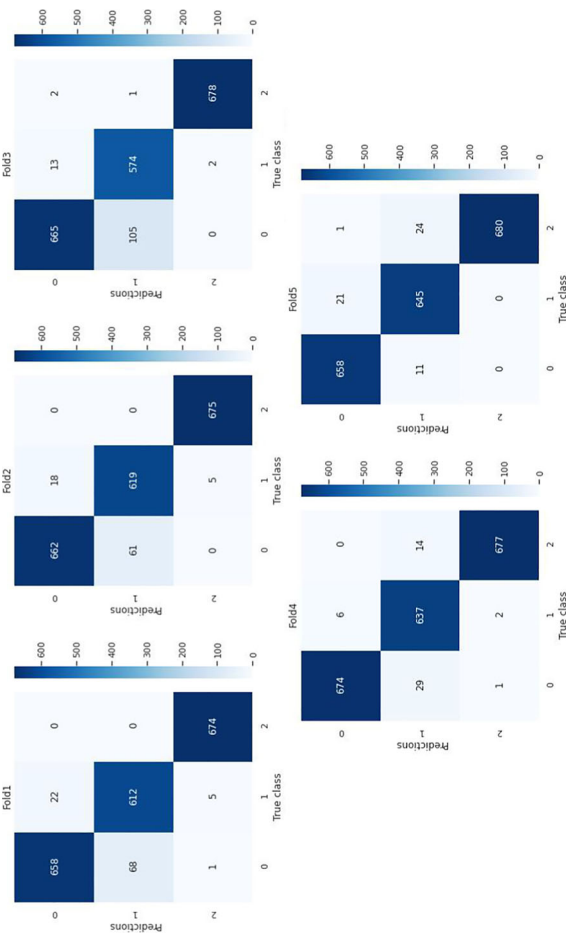


Fig. 14 Comparison of the test accuracy of OMRNet and post quantized OMRNet

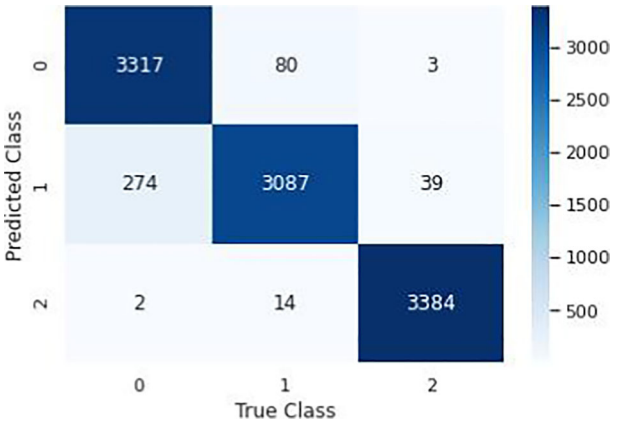


Fig. 15 Confusion matrices of the 5 folds

From the confusion matrices, it is clearly evident that our model does a good job of identifying empty images. Out of 10200 images (3400 per class), our model correctly classifies -

- 3384 images of empty class (99.52 % accuracy)
- 3087 images of crossed out class (90.79 % accuracy)
- 3317 images of confirmed class (97.55 % accuracy)

Overall, our model attains an average accuracy of 95.96 %. The model performs a bit poor while classifying between confirmed and crossed out classes, as it misclassifies about 8% of the confirmed class to be crossed out and about 2.35% of the crossed out class as confirmed. Figure 9 shows some of the correctly classified images.

4.8.1 Error case analysis

This section analyses the standard error metrics of our model and some of the images which our model misclassifies. A CNN model performs best when the classes of the dataset have well defined features. As evident from Fig. 17, the dataset that we have used for our model contains a large amount of inter-class similarities and intra-class variances. This is one of the main reasons for the misclassifications done by OMRNet. Apart from this, in many cases, the ROIs of the dataset are not properly extracted. This also causes OMRNet to misclassify some of the images.

The precision and recall scores (in Table 13) clearly reflect that our model does not perform very well in classifying between crossed out and confirmed images. It misclassifies a total of 274 out of 3400 confirmed cases as crossed out and 80 crossed out cases out of 3400 crossed out cases as confirmed. Some of the reasons for incorrectly classified images can be seen in Fig. 17. As is evident from this discussion, most of the erroneously recognized answer box images either have very similar features as another class (usually between 0 and 1) or the ROI in the dataset is not properly extracted (usually for 2).



Fig. 16 The confusion matrix summed over all 5 folds

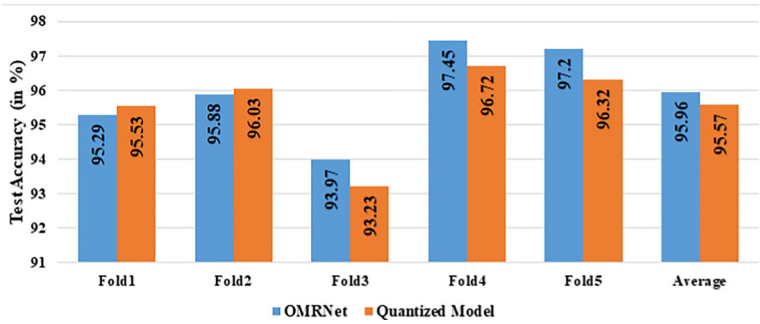


Fig. 17 Some misclassified answer box images from each class

These misclassified results are probably caused by our model being unable to accommodate all the nuances of the dataset. There are many images wherein the pattern of crossing out an answer box is the same as confirming an answer box due to a difference in the templates. For our model to successfully identify all those features, without overfitting, a broader and more diverse dataset would be required. Currently, our model does significantly better than previous research attempts made in this field, as discussed in the Section 4.9.

4.9 Comparison with past methods

The 5-fold cross-validation accuracy of the CNN model described in [1] is 92.70% for confirmed, crossed out, and empty classification. The proposed OMRNet architecture gives an accuracy of 95.96% for the same 5-fold cross validation and can prove effective in replacing the existing CNN model in [1]. As our model improves the classification accuracy by around 3.26%, it should help in improving the overall classification accuracy in conjunction with the BoVW classifier as a 2-stage ensemble classifier, as mentioned in [1]. The results are further summarized in Table 14

4.10 Advantages and limitations

The advantages of the proposed model are listed below:

- OMRNet has achieved an accuracy of 95.96% on the MC Answer Boxes Dataset used here. To the best of our knowledge, the performance of OMRNet is 4% better than state-of-the-art models.
- It is a single stage architecture unlike two-stage ones proposed by many past researchers.
- The model is extremely lightweight, and it has the best execution time among its contemporaries.

Table 13 Precision, Recall, and F1 Score of the OMRNet model across classes

Metric	Confirmed	Crossed out	Empty
Precision	0.9251	0.9705	0.9880
Recall	0.9756	0.9079	0.9953
F1 Score	0.9492	0.9378	0.9916

Table 14 Comparison of the OMRNet with other models in terms of average accuracy while classifying an answer box into confirmed, crossed-out, and empty classes across the 5-folds

Model	Average Accuracy
Various Models proposed by Mahmoud Afifi & Khaled F. Hussain [1]	
Baseline	60.4 %
NBC	29.3%
BoVW	90.9%
CNN (1s) (using AlexNet)	92.7 %
OMRNet (Proposed)	95.96%

– Use of quantization algorithms has significantly reduced the model size, thereby making it an ideal candidate for lightweight edge devices.

A few limitations of the proposed work are as follows:

– Currently our model is trained on one dataset. Hence, it may not be generic.
– Although we have achieved a very high accuracy of 95.96% yet there is still scope for improvement.
– This model relies on answer boxes to give a verdict. An end-to-end model can be trained to predict the confirmed one from the choices set instead of relying on individual answer boxes.
– We have followed a trial-and-error method to select the CNN model. One can use Neural Architecture Search (NAS) [64] to select a neural network best suited for the underlying task.

5 Conclusion and future work

The existing OMR techniques, in general, use hard-coded image processing algorithms. This hampers the flexibility of the system and also demands extra caution while marking and handling the sheet. A few systems exist that rely on deep neural networks to classify answer boxes but they are not enough lightweight to be suitable for resource constraint environments. These combined with the need to remove the rigidity in the correction of MCQ tests motivated us to develop our proposed model. To this end, we propose a lightweight CNN model, called OMRNet, which performs reasonably well in classifying answer boxes without compromising on performance.

OMRNet provides high accuracy (95.96 %) on a complex and generalized dataset and is seen to be the best performing model of its class in the domain of flexible generalized OMR techniques according to the best of our knowledge. However, it inherits some of the deficiencies of a CNN classifier and performs poorly in distinguishing between classes with a large number of similar features. Future work would include building a complete mobile app, where on insertion of an OMR sheet with its answer key, the system gives back the score of the test giver and its use in practice. This will lead to the collection of more real-world data and hence diversify the original dataset, allowing us to use the concepts of data-driven development and MLOps [38]. Consequently, training and tuning our model on that diverse dataset would lead to better predictive accuracy. In a problem such as the one our paper tries to solve, there is no room for error. A single wrongly classified answer box can be the difference between a student passing, and failing. It can make or break a career, especially in

highly competitive examinations. Thus our quest for achieving cent percent accuracy shall continue.

Acknowledgements We are thankful to the Center for Microprocessor Applications for Training Education and Research (CMATER) research laboratory of the Computer Science and Engineering Department, Jadavpur University, Kolkata, India for providing infrastructural support.

Data Availability The data used in this work is publicly available and it can be downloaded from the link: https://sites.google.com/view/mcq-dataset.

Code Availability The code can be accessed using the link: https://github.com/sa-y-an/OMRNet.

Declarations

Conflict of Interest Statement The authors declare that they have no conflict of interest.

References

1. Afifi M, Hussain K (2019) The achievement of higher flexibility in multiple-choice-based tests using image classification techniques. International Journal On Document Analysis And Recognition (IJDAR) 22:127–142
2. Agarap A (2018) Deep learning using rectified linear units (ReLU). arXiv:1803.08375
3. Agarap A (2018) Deep learning using rectified linear units (ReLU). arXiv:1803.08375
4. Ahad, R, Toufiq, R, Zaman S (2019) Information retrieval from damage OMR sheet by using image processing. In: 2019 3rd International Conference On Electrical, Computer & Telecommunication Engineering (ICECTE), pp 153–156
5. Al-marakeby A (2013) Multi core processors for camera based OMR. Int J Comput Appl 68
6. Anvarjon, T, Mustaqeem, Kwon S (2020) Deep-Net: A lightweight CNN-based speech emotion recognition system using deep frequency features. Sensors 20, https://www.mdpi.com/1424-8220/20/18/5212
7. Bera S, Kar R, Saha S, Chakrabarty A, Lahiri S, Malakar S, Sarkar R (2018) A one-pass approach for slope and slant estimation of tri-script handwritten words. J Intell Syst 29:688–702
8. Bhowal, P, Banerjee, D, Malakar, S, Sarkar R (2022) A two-tier ensemble approach for writer dependent online signature verification. J Ambient Intell Human Comput, pp 1–20
9. Buleje, C, Atencio, Y, Tinoco E (2020) System with optical mark recognition based on artificial vision for the processing of multiple selection tests in school competitions. In: 2020 XLVI Latin American Computing Conference (CLEI), pp 172–177
10. Calado, M, Ramos, A, Jonas P (2019) An application to generate, correct and grade multiple-choice tests. In: 2019 6th International Conference On Systems And Informatics (ICSAI), pp 1548–1552
11. Chai D (2016) Automated marking of printed multiple choice answer sheets. In: 2016 IEEE International conference on Teaching, Assessment, and Learning for Engineering (TALE), pp 145–149
12. Chattopadhyay S, Dey A, Singh P, Geem Z, Sarkar R (2021) COVID-19 detection by optimizing deep residual features with improved clustering-based golden ratio optimizer. Diagnostics 11:315
13. China, R, DeAssisZampirolli, F, DeOliveiraNeves, R, Quilici-Gonzalez J (2016) An application for automatic multiple-choice test grading on android. Revista Brasileira De IniciaÇÃo Científica 3
14. Chinnasarn, K, Rangsanseri Y (1999) Image-processing-oriented optical mark reader. In: Applications of digital image processing XXII, pp 702–708
15. Das, S, Roy, S, Malakar, S, Velásquez, J, Sarkar R (2021) Bi-level prediction model for screening COVID-19 patients using chest X-Ray images. Big Data Research, p 100233
16. Deng, J, Dong, W, Socher, R, Li, L, Li, K, Fei-Fei L (2009) Imagenet: A large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition, pp 248–255
17. Dey M, Mia S, Sarkar N, Bhattacharya A, Roy S, Malakar S, Sarkar R (2021) A two-stage CNN-based hand-drawn electrical and electronic circuit component recognition system. Neural Comput Appl 33:13367–13390
18. Dey S, Roychoudhury R, Malakar S, Sarkar R (2022) An optimized fuzzy ensemble of convolutional neural networks for detecting tuberculosis from Chest X-ray images. Appl Soft Comput 114:108094

19. Elias E, Tasinaffo P, Hirata R Jr (2021) Optical mark recognition: Advances, difficulties, and limitations. SN Comput Sci 2:367
20. Fisteus J, Pardo A, García N (2013) Grading multiple choice exams with low-cost and portable computer-vision techniques. J Sci Educ Technol 22:560–571
21. Gillespie, R, Michael, M, Sokal (ed) (1988) Psychological testing and american society, 1890–1930. New Brunswick, NJ: Rutgers University Press, 1987. Pp. xi+ 205. ISBN 0-8135-1193-3. $28.00.. The british journal for the history of science. 21, pp 118–119
22. Gyamfi E, Missah Y (2017) Pixel-based unsupervised classification approach for information detection on optical markup recognition sheet. Adv Sci Technol Eng Syst J 2:121–132
23. He, K, Zhang, X, Ren, S, Sun J (2016) Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp 770–778, https://doi.org/10.1109/CVPR.2016.90
24. Howard, A, Sandler, M, Chu, G, Chen, L, Chen, B, Tan, M, Wang, W, Zhu, Y, Pang, R, Vasudevan, V, et al (2019) Searching for mobilenetv3. In: Proceedings of the IEEE/CVF International conference on computer vision, pp 1314–1324
25. Huang, G, Liu, Z, VanDerMaaten, L, Weinberger K (2017) Densely connected convolutional networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 4700–4708
26. Hussmann S, Deng P (2005) A high-speed optical mark reader hardware implementation at low cost using programmable logic. Real-Time Imaging 11:19–30
27. Jacob, B, Kligys, S, Chen, B, Zhu, M, Tang, M, Howard, A, Adam, H, Kalenichenko D (2018) Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: Proceedings of the IEEE Conference on computer vision and pattern recognition, pp 2704–2713
28. Jain, V, Malik, S, Bhatia V (2022) Robust image processing based real-time optical mark recognition system. In: 2022 IEEE 6th Conference On Information And Communication Technology (CICT), pp 1–5
29. Jingyi, T, Hooi, Y, Bin O (2021) Image processing for enhanced OMR answer matching precision. In: 2021 International Conference On Computer & Information Sciences (ICCOINS), pp 322–327
30. Kamavisdar P, Saluja S, Agrawal S (2013) A survey on image classification approaches and techniques. Int J Adv Res Comput Commun Eng 2:1005–1009
31. Karunanayake N (2015) OMR sheet evaluation by web camera using template matching approach. Int J Res Emerg Sci Technol 2
32. Khan, I, Rahman, S, Alam F (2018) An efficient, cost effective and user friendly approach for MCQs treatment
33. Kingma, D, Ba J (2017) Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations, ICLR 2015,San Diego, CA, USA, May 7-9, 2015, conference track proceedings
34. Kolesnikov, A, Beyer, L, Zhai, X, Puigcerver, J, Yung, J, Gelly, S, Houlsby N (2020) Big transfer (bit): General visual representation learning. In: European conference on computer vision, pp 491-507
35. Koushik, K, Sourav Chengappa, B, Chendan R (2022) Automated marks entry processing in Handwritten answer scripts using character recognition techniques. In: 2022 3rd International Conference On Electronics And Sustainable Communication Systems (ICESC), pp 728–733
36. KRISNADI, D, MITRA, A, DESANTI, R, CIPUTRA, W, Hery H (2017) A multiple-choice test recognition system based on android and RBFNN. DEStech Transactions On Computer Science And Engineering
37. Kuklick H (1987) The testing movement and its founders. Sci 237:1358–1359
38. Mäkinen, S, Skogström, H, Laaksonen, E, Mikkonen T (2021) Who needs MLOps: What data scientists seek to accomplish and how can MLOps help?. arXiv:2103.08942
39. Malakar, S, Sahoo, S, Chakraborty, A, Sarkar, R, Nasipuri M (2022) Handwritten arabic and roman word recognition using holistic approach. Vis Comput, pp 1–24
40. Malakar, S, Seraogi, B, Sarkar, R, Das, N, Basu, S, Nasipuri M (2012) Two-stage skew correction of handwritten Bangla document images. In: 2012 3rd International conference on emerging applications of information technology, pp 303–306
41. Malakar S, Ghosh M, Sarkar R, Nasipuri M (2018) Development of a two-stage segmentation-based word searching method for handwritten document images. J Intell Syst 29:719–735
42. Marchant J (2021) Assessing the validity of multiple-choice questions, using them to undertake comparative analysis on student cohort performance, and evaluating the methodologies used
43. Mondal, R, Malakar, S, Barney Smith, E, Sarkar R (2022) Handwritten english word recognition using a deep learning based object detection architecture. Multimed Tools Appl, pp 1–26
44. Mustaqeem, Kwon S (2020) A CNN-assisted enhanced audio signal processing for speech emotion recognition. Sensors 20
45. Mustaqeem Kwon S (2021) Att-Net: Enhanced emotion recognition system using lightweight self-attention module. Appl Soft Comput 102:107101

46. Nguyen, T, Manh, Q, Minh, P, Thanh, L, Hoang T (2011) Efficient and reliable camera based multiple-choice test grading system. In: The 2011 International Conference on Advanced Technologies for Communications (ATC 2011), pp 268–271
47. Pavithra, U, Sneha, MR, Sagar Srivatsa SR, Ravichandra, T, Aparanji VM (2019) OCR system for automating answer script Marks using auto resonance network. In: 2019 International Conference On Communication And Signal Processing (ICCSP), pp 0857–0861
48. Rasiq, G, AlSefat, A, Hasnain M (2019) Mobile based MCQ answer sheet analysis and evaluation application. In: 2019 8th International conference System Modeling and Advancement in Research Trends (SMART), pp 144–147
49. Sandler, M, Howard, A, Zhu, M, Zhmoginov, A, Chen L-C, Ler, M, Howard, A, Zhu, M, Zhmoginov, A, Chen L (2018) Mobilenetv2: Inverted residuals and linear bottlenecks. In: 2018 IEEE/CVF Conference on computer vision and pattern recognition, pp 4510-4520, https://doi.org/10.1109/CVPR.2018.00474
50. Sanguansat P (2015) Robust and low-cost Optical Mark Recognition for automated data entry. 2015 12th International Conference On Electrical Engineering/Electronics. Computer, Telecommunications And Information Technology (ECTI-CON), pp 1–5
51. Seidaliyeva U, Akhmetov D, Ilipbayeva L, Matson E (2020) Real-time and accurate drone detection in a video with a static background. Sensors 20:3856
52. Shaikh, E, Mohiuddin, I, Manzoor, A. Latif, G, Mohammad N (2019) Automated grading for handwritten answer sheets using convolutional neural networks. In: 2019 2nd International Conference on new Trends in Computing Sciences (ICTCS), pp 1–6
53. Simonyan, K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv:1409.1556
54. Sinchai, A, Tuwanut P (2022) Using of an arithmetic sequence to estimate undetected existing circle choice locations. In: 2022 37th International Technical Conference On Circuits/Systems, Computers And Communications (ITC-CSCC), pp 268–271
55. Smith A (1981) Optical mark reading - Making it easy for users. In: Proceedings of the 9th annual ACM SIGUCCS conference on User services'81, pp 257–263
56. Spadaccini, A, Rizzo V (2011) A multiple-choice test recognition system based on the gamera framework. arXiv:1105.3834
57. Supic, M, Brkic, K, Hrkac, T, Mihajlović, Kalafatić Z (2014) Automatic recognition of handwritten corrections for multiple-choice exam answer sheets. In: 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), pp 1136–1141
58. Szegedy, C, Ioffe, S, Vanhoucke, V, Alemi A (2017) Inception-v4, inception-resnet and the impact of residual connections on learning. In: Thirty-first AAAI Conference on artificial intelligence
59. Tan, M, Le Q (2019) Efficientnet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning, pp 6105–6114
60. The University of Texas "Alumni Notes" (1973) The alcalde: The university of texas at austin alumni magazine. 61(5):36. ISSN 1535-993X. Retrieved 29 November 2020
61. Weaver, D, Chalkley B (1997) Introducing objective tests and OMR-based student assessment A case study. Journal Of Geography In Higher Education
62. Wolberg G (1990) Digital image warping. IEEE computer society press Los Alamitos, CA
63. Zhu M, Min W, Wang Q, Zou S, Chen X (2021) PFLU and FPFLU: Two novel non-monotonic activation functions in convolutional neural networks. Neurocomputing 429:110–117
64. Zoph, B, Le Q (2016) Neural architecture search with reinforcement learning. arXiv:1611.01578

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Sayan Mondal[1] · Pratyay De[1] · Samir Malakar[2] · Ram Sarkar[3]

Sayan Mondal
dgpmondal@gmail.com

Pratyay De
pratyaydedgp@gmail.com

Ram Sarkar
ramjucse@gmail.com

1 Department of Electrical Engineering, National Institute of Technology Durgapur, Durgapur 713209, India

2 Department of Computer Science, Asutosh College, Kolkata 700026, India

3 Department of Computer Science and Engineering, Jadavpur University, Kolkata 700032, India